
Departamento de Energía y Automatización
Robótica Industrial

PRÁCTICA Nº 2

Sensores

Asignatura: Robótica Industrial (FPTEN27)

Profesores: José Barriola · Leonardo Delgado · José Montilva

Guía de Práctica de Laboratorio

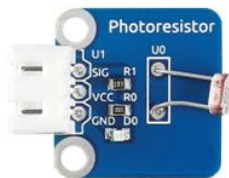
Práctica #2 — Sensores

Objetivo General

Familiarizarse con el uso de sensores digitales y analógicos conectados a Arduino, comprendiendo cómo cada tipo de señal (Di, Do, Ai, PWM) interactúa con el hardware, y explorar técnicas de temporización no bloqueante mediante `millis()`.

Ejercicio #1 Sensor de Luz (LDR)

El sensor de luz utilizado es una fotorresistencia (LDR, del inglés Light Dependent Resistor). Se trata de un componente cuya resistencia eléctrica disminuye a medida que aumenta la intensidad de la luz que recibe — fenómeno conocido como fotoconductividad. Esto lo hace útil para detectar condiciones de luz y oscuridad en circuitos automatizados.



Módulo fotorresistencia (LDR)



PCB interna del módulo — comparador LM393 y potenciómetro de umbral

Principio de operación

El módulo LDR incluye internamente un comparador analógico (LM393). La señal que llega al pin SIG es analógica: varía entre 0 y 1023 según la cantidad de luz. En este ejercicio, se conecta ese pin al A0 del Arduino y se usa `analogRead()` para leer ese valor. Cuando el valor supera 200 (umbral configurable en el código), el LED del pin 13 se enciende; de lo contrario, se apaga.

⚠ Nota pedagógica — Umbral ajustable

- El valor 200 es un umbral inicial. Dependiendo de las condiciones de iluminación del laboratorio, puede que necesites modificarlo.
- Esto refleja un problema real en sistemas embebidos: los umbrales fijos fallan en entornos con luz variable.
- Reflexiona: ¿cómo harías ese umbral más robusto si la luz varía constantemente?

Programa

```
int valorLDR = 0;      // Almacena el valor leído del LDR
int pinLed1  = 13;     // Pin del LED de salida
int pinLDR   = A0;     // Pin analógico del sensor

void setup() {
  pinMode(pinLed1, OUTPUT);
  Serial.begin(9600);  // Monitor serial para ver los valores
}

void loop() {
  valorLDR = analogRead(pinLDR);  // Lee señal analógica (0-1023)
  Serial.println(valorLDR);        // Imprime en Monitor Serial

  if (valorLDR >= 200) {
    digitalWrite(pinLed1, HIGH);
  } else {
    digitalWrite(pinLed1, LOW);
  }
}
```

⚠ ¿Qué observar durante el ejercicio?

- Abre el Monitor Serial (Ctrl+Shift+M) y observa los valores mientras tapas el sensor con la mano.
- Anota el valor aproximado con luz ambiente y el valor con el sensor tapado — esto te permitirá comprender el rango de operación del sensor.

Ejercicio #2 Sensor Infrarrojo — Seguidor de Línea (TCRT5000)

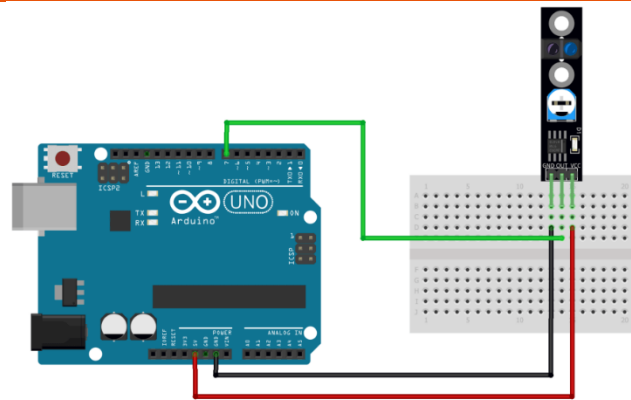
El módulo TCRT5000 es un sensor de reflexión infrarroja. Emite luz infrarroja de forma continua y mide cuánta luz rebota de vuelta. Las superficies blancas reflejan más luz (alta reflectancia), mientras que las superficies negras la absorben (baja reflectancia). Esta diferencia determina el estado lógico que entrega el sensor al Arduino.

Principio de operación

El módulo incluye un comparador interno. Cuando la luz reflejada supera un umbral (ajustable con el potenciómetro integrado), el pin SIG emite LOW; si la reflexión es baja, emite HIGH. La lógica puede parecer invertida — presta atención a este detalle.



Módulo IR (TCRT5000) — vista de componentes



fritzing

Conexión: SIG → pin 7 del Arduino UNO

Programa

```

int sensorPin = 7;    // Pin digital de salida del sensor IR
int ledPin     = 13;   // LED indicador integrado del Arduino
int val;          // Variable para almacenar la lectura

void setup() {
  pinMode(sensorPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  val = digitalRead(sensorPin); // Lee HIGH o LOW

  if (val == HIGH) {
    Serial.println("Linea detectada");
    digitalWrite(ledPin, LOW);
  } else {
    Serial.println("Linea NO detectada");
    digitalWrite(ledPin, HIGH);
  }
  delay(500);
}

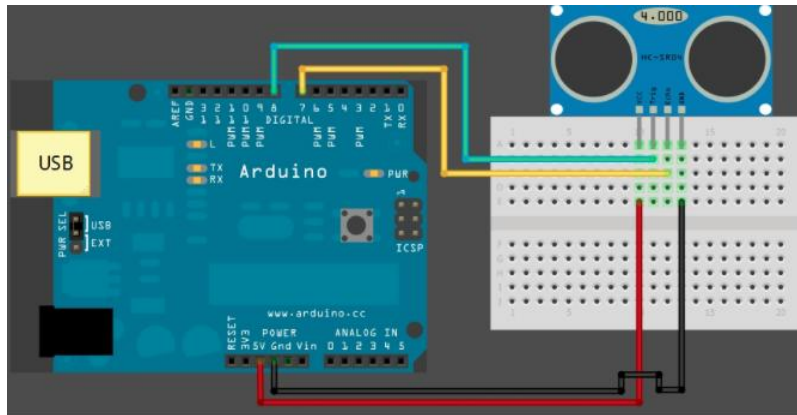
```

⚠ ¿Qué observar y ajustar?

- Prueba el sensor sobre papel blanco y luego sobre una línea negra (cinta o marcador) — observa tanto el Monitor Serial como el LED del pin 13.
- Si el sensor no distingue bien entre blanco y negro, gira el potenciómetro azul del módulo — este ajusta el umbral de comparación interna.
- Trata de entender qué parámetro interno del sensor se está ajustando al girar el potenciómetro — es parte fundamental del principio de operación del módulo.

Ejercicio #3 Sensor Ultrasónico HC-SR04 — Medición de Distancia

El HC-SR04 mide distancias usando pulsos de sonido de alta frecuencia (40 kHz, inaudibles). Funciona emitiendo un pulso corto por el pin TRIG y midiendo el tiempo que tarda en llegar el eco por el pin ECHO. A partir de ese tiempo se calcula la distancia, dado que la velocidad del sonido en el aire es conocida.



Conexión HC-SR04: TRIG → pin 8, ECHO → pin 7, VCC → 5V, GND → GND

¿Cómo calcula la distancia el código?

La fórmula $\text{distancia} = \text{duración} / 58.2$ viene de la física del sonido. La velocidad del sonido a temperatura ambiente es aproximadamente 343 m/s. El pulso viaja hasta el objeto y regresa (doble distancia), por lo que: $\text{distancia [cm]} = \text{tiempo [\mu s]} / 58.2$. Este valor de 58.2 ya incorpora el factor de ida y vuelta.

Programa

```

#define echoPin 7          // Pin ECHO
#define trigPin 8          // Pin TRIG
#define LEDPin 13          // LED indicador

int maximoRango = 200;
int minimoRango = 0;
long duracion, distancia;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(LEDPin, OUTPUT);
}

void loop() {
  // 1. Disparo: pulso de 10 μs en trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // 2. Medición: esperar eco y calcular distancia
  duracion = pulseIn(echoPin, HIGH);

```

```

distancia = duracion / 58.2;

// 3. Validación de rango
if (distancia >= maximoRango || distancia <= minimoRango) {
  Serial.println("-1");
  digitalWrite(LEDpin, HIGH);
} else {
  Serial.print("DISTANCIA = ");
  Serial.print(distancia);
  Serial.println(" Cm");
  digitalWrite(LEDpin, LOW);
}
delay(50);
}

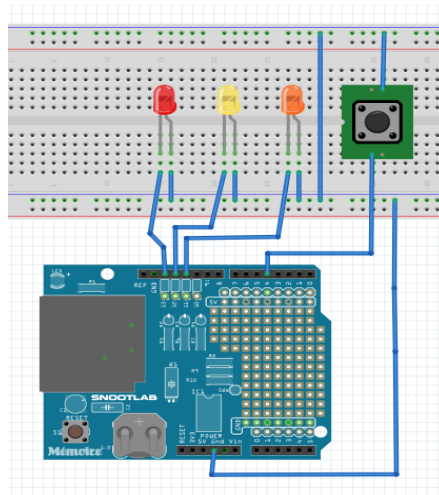
```

⚠ Notas de operación y limitaciones del sensor

- El rango útil es de 2 cm a 300 cm. A menos de 2 cm el sensor no puede leer correctamente — el eco regresa antes de que el pin ECHO esté listo.
- La función `pulseIn()` es BLOQUEANTE: el código se detiene y espera el eco. Si no hay objeto, espera hasta que se agote un timeout interno.
- Verifica la distancia midiendo con una regla — compara con lo que muestra el Monitor Serial.
- Comprende la relación entre la fórmula $/58.2$ y la física del sonido — es el vínculo entre la medición temporal y la distancia real.

Ejercicio #4 Temporización No Bloqueante — `delay()` vs. `millis()`

Este ejercicio demuestra un problema fundamental en sistemas embebidos: el uso de `delay()` bloquea completamente el procesador. Mientras Arduino espera, no puede responder a botones, sensores ni comunicación serial. La función `millis()` permite medir tiempo sin detener la ejecución del programa.



Circuito: LED1 (pin 13), LED2 (pin 12), LED3 (pin 11), Botón (pin 4 con INPUT_PULLUP)

Parte A — Código con delay()

Cargue y ejecute este primer programa. Verifique si las cuatro tareas funcionan simultáneamente de forma correcta:

```
#define LED1 13    // Parpadea cada 500 ms
#define LED2 12    // Enciende al presionar botón
#define LED3 11    // Brillo variable por Serial
#define BTN 4

int LED1_state = LOW;
int brightness = 0;

void setup() {
  Serial.begin(9600);
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);
  pinMode(BTN, INPUT_PULLUP); // Botón activo en LOW
}

void loop() {
  // Tarea 1: Blink LED1 cada 500 ms (usa delay)
  LED1_state = !LED1_state;
  delay(500);
  digitalWrite(LED1, LED1_state);
  delay(500);

  // Tarea 2: Botón → LED2
  if (digitalRead(BTN)) {
    digitalWrite(LED2, LOW);
  } else {
    digitalWrite(LED2, HIGH);
  }

  // Tarea 3: Serial → brillo LED3
  if (Serial.available()) {
    brightness = Serial.parseInt();
    if (brightness >= 0 && brightness <= 255) {
      analogWrite(LED3, brightness);
      delay(5000); // ← BLOQUEA todo el programa 5 s tras recibir un valor
    }
  }

  // Tarea 4: Imprime brillo cada 5 segundos
  // NOTA: este delay(5000) está FUERA del if(Serial.available()),
  // por lo que se ejecuta en CADA iteración del loop,
  // bloqueando el programa 5 s continuamente sin importar el botón.
  Serial.print("Brightness (0-255): ");
  Serial.println(brightness);
  delay(5000);
}
```

⚠ ¿Qué falla en esta versión?

- Hay dos fuentes de bloqueo: el delay(5000) de Tarea 3 (solo ocurre si recibes un valor) y el delay(5000) de Tarea 4, que bloquea el loop SIEMPRE, en cada iteración.
- Presiona el botón en cualquier momento — el LED2 no responde hasta que termina el delay activo.
- El LED1 debería parpadear cada 500 ms, pero los delays adicionales lo desincronizarán completamente.
- Documenta tus observaciones con detalle — comprender por qué ocurre este comportamiento es el objetivo central del ejercicio.

Parte B — Código con millis()

Ahora cargue la versión mejorada. Compara el comportamiento del botón y del parpadeo:

```
#define LED1 13
#define LED2 12
#define LED3 11
#define BTN 4

int LED1_state = LOW;
int brightness = 0;

unsigned long prevTime_T1 = millis();
unsigned long prevTime_T4 = millis();
long interval_T1 = 1000; // Blink cada 1 segundo
long interval_T4 = 5000; // Imprime cada 5 segundos

void setup() {
  Serial.begin(9600);
  pinMode(LED1, OUTPUT); pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT); pinMode(BTN, INPUT_PULLUP);
}

void loop() {
  unsigned long currentTime = millis();

  // Tarea 1: Blink sin bloquear
  if (currentTime - prevTime_T1 > interval_T1) {
    LED1_state = !LED1_state;
    digitalWrite(LED1, LED1_state);
    prevTime_T1 = currentTime;
  }

  // Tarea 2: Botón — ahora sí responde inmediatamente
  if (digitalRead(BTN)) {
    digitalWrite(LED2, LOW);
  } else {
    digitalWrite(LED2, HIGH);
  }

  // Tarea 3: Serial → brillo
  if (Serial.available()) {
    brightness = Serial.parseInt();
    if (brightness >= 0 && brightness <= 255)
      analogWrite(LED3, brightness);
  }
}
```

```
// Tarea 4: Imprime brillo sin bloquear
if (currentTime - prevTime_T4 > interval_T4) {
  Serial.print("Brightness (0-255): ");
  Serial.println(brightness);
  prevTime_T4 = currentTime;
}
}
```

⚠ ¿Qué observar en la versión con millis()?

- Presiona el botón en cualquier momento — ahora el LED2 debe responder de inmediato.
- El LED1 parpadea de forma regular sin importar qué más esté haciendo el programa.
- La clave: `if (currentTime - prevTime > interval)` actúa como un reloj sin pausar el procesador.